

TASK 1: FACTORY METHOD

1.5 (6 marks) - Why Factory Method is better than if/else

Factory Method is better than a `createConnector(string kind)` function with if/else chains because it follows the Open-Closed Principle.

With if/else, every time you add a new connector type (like a new database), you must modify the existing `createConnector()` function. This violates the principle because the code is not closed for modification.

With Factory Method, you simply create a new concrete factory class (e.g., `MySQLFactory`) without changing any existing code. The code is open for extension (add new factories) but closed for modification (don't change existing factories). This makes the system more maintainable and scalable.

1.6 (8 marks) - Factory Method Participants

- `Connector` = Product (abstract interface for all connectors)
- `CsvConnector` = ConcreteProduct (specific implementation that inherits from `Connector`)
- `ConnectorFactory` = Creator (abstract factory class with abstract `createConnector()` method)
- `CsvFactory` = ConcreteCreator (concrete factory that overrides `createConnector()` to return `CsvConnector`)

TASK 2: PROTOTYPE PATTERN

2.5 (6 marks) - Shallow vs Deep Copy

A shallow copy copies references to object data but not the data itself. If the original has a pointer member variable pointing to heap memory, the copy will point to the SAME heap memory. Changes to one affect the other, and deleting one causes the other to have a dangling pointer.

A deep copy creates independent copies of all data, including any heap-allocated memory. Each object has its own separate memory. The `clone()` method must produce a deep copy so that each cloned transformation step is completely independent of the prototype.

The clone must be independent because multiple transformation steps can be active simultaneously in the pipeline. If they shared memory, modifying one step's state would corrupt others.

2.6 (6 marks) - Why Registry Returns Clones

The registry returns `clone()` of the stored prototype rather than the stored pointer itself because the registry owns the prototypes and must keep them unchanged.

If two pipelines shared one prototype, they would both hold pointers to the same object. When one pipeline's transform() step modifies records or state, it would affect the other pipeline's step. This breaks the independence of separate pipeline runs.

By cloning, each pipeline gets its own separate copy of the transformation step. The registry keeps the original prototypes pristine and reusable.

TASK 3: TEMPLATE METHOD PATTERN

3.5 (6 marks) - Why run() is NOT Virtual

run() is deliberately not virtual because it defines the fixed lifecycle (template method). If a subclass could override run(), it could break the structure by changing the order of steps, skipping steps, or adding new steps in arbitrary places.

By making run() non-virtual, we force all subclasses to follow the same order: connect() → extract() → transform() → load(). Subclasses can only override extract() and load(), the points where their behavior legitimately differs.

3.6 (8 marks) - Pipeline Methods Classification

Template method: run() - defines the fixed skeleton of the algorithm

Concrete steps (same for all pipelines):

- connect() - gets a connector and prints output
- transform() - applies all transformation steps in order

Primitive operations (overridden by subclasses):

- extract() - different for BatchPipeline vs StreamingPipeline
- load() - different for BatchPipeline vs StreamingPipeline

Factory Method and Template Method do NOT overlap in Pipeline. Factory Method creates the connector objects without naming concrete classes. Template Method defines the fixed order of operations (connect→extract→transform→load) that all pipelines follow. They address different concerns: object creation vs. algorithm structure.

TASK 4: MEMENTO PATTERN

4.4 (6 marks) - Narrow and Wide Interfaces

A Memento has two interfaces to protect encapsulation:

The narrow interface (what Caretaker sees) contains only save() and undo() methods. The CheckpointManager can only pass the checkpoint between objects - it cannot look inside or access the captured state. This prevents the caretaker from accidentally modifying or depending on the internal state.

The wide interface (what Originator sees) contains `getStage()` and `getRecords()`. Only Pipeline (the originator that created the checkpoint) can access these methods to restore its state. This allows the originator to fully restore itself while preventing other objects from accessing sensitive state data.

4.5 (8 marks) - Failure Recovery Scenario

Step-by-step recovery process:

1. Pipeline runs `connect()` → `extract()` → `transform()` → `load()` successfully up to stage 3.
2. At the end of `run()`, Pipeline calls `createCheckpoint()` to capture current stage and records.
3. CheckpointManager calls `save(checkpoint)` to store this `RunCheckpoint` in its history vector.
4. Later at 02:00, the pipeline fails during `load()` (stage 4).
5. To resume, CheckpointManager calls `undo()` to retrieve the last checkpoint from history.
6. Pipeline calls `restore(checkpoint)` to set stage and records back to the captured values.
7. The pipeline is now at stage 3 with all the records that had been processed, ready to resume `load()` without repeating previous work.

4.6 (6 marks) - Memento Participants

- Pipeline = Originator (creates checkpoints holding its internal state: stage and records)
- RunCheckpoint = Memento (the snapshot object that stores the originator's state)
- CheckpointManager = Caretaker (holds and manages the collection of checkpoints without accessing their contents)